

EXPERIMENT NO. 11 / 12

White-Box Testing & Cyclomatic Complexity Analysis

Project: Hospital Management System (HMS)

Prepared by: Anish Chitnis (24CE1044) | Aaryan Choube (24CE1045) | Somansh Dafade (24CE1047)
Batch: A / A1 | Dept of Computer Engineering, RAIT, DY Patil University

1. Aim

To develop test cases for the Hospital Management System (HMS) using White-Box Testing (Basis Path Testing), construct the Control Flow Graph (CFG), compute McCabe's Cyclomatic Complexity, and achieve 100% statement and branch coverage on the calculateBill() method.

2. Objectives

- To analyze source code logic and construct a Control Flow Graph (CFG).
- To compute McCabe's Cyclomatic Complexity $V(G)$ using all three standard mathematical methods.
- To derive linearly independent basis paths.
- To design test cases with inputs and expected outputs to execute every basis path.

3. Method Under Test: calculateBill()

```
public class HospitalBillingTest {  
    public static double calculateBill(double fee, double labCost, boolean hasIns, boolean  
isEmerg) {  
        if (fee == 0) {                                // Node 1 (Predicate 1)  
            return -1.0;                                // Node 2 (Error Return)  
        }  
        double totalBill = fee + labCost;                // Node 3 (Sequential Statement)  
        if (hasIns || (isEmerg && totalBill > 5000)) {    // Node 4 (Predicate 2)  
            totalBill = totalBill * 0.90;                // Node 5 (Apply 10% Discount)  
        }  
        return totalBill;                                // Node 6 (Return Final Bill)  
    }                                                    // Node 7 (Method Exit)  
}
```

4. Control Flow Graph (CFG)

Control Flow Graph (CFG) - calculateBill() Method

Nodes N = 7, Edges E = 8, Connected Components P = 1 $\Rightarrow V(G) = 8 - 7 + 2(1) = 3$

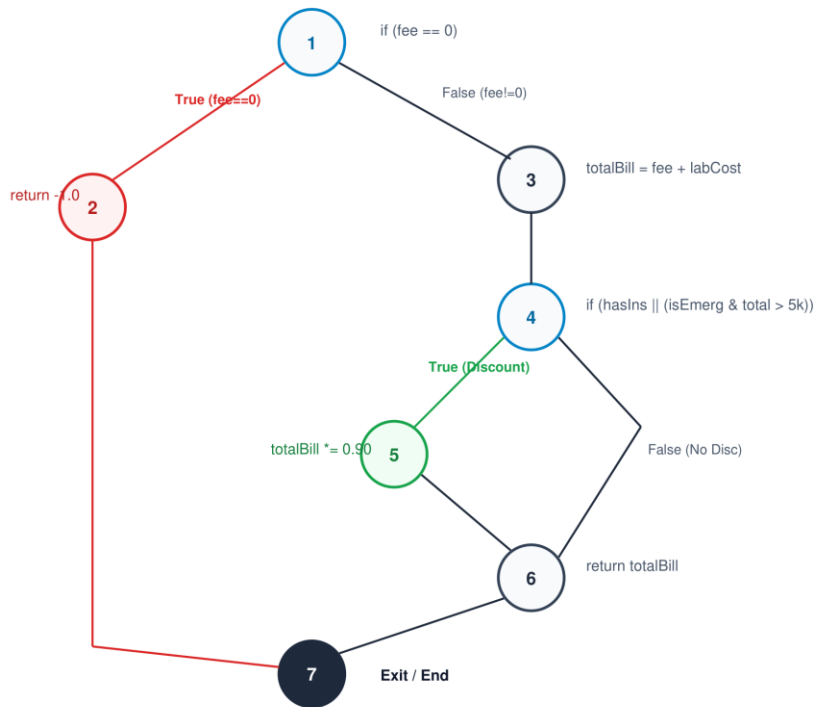


Fig 1: Control Flow Graph (CFG) of calculateBill() Method

5. Cyclomatic Complexity Computation

From the CFG:

- Total Nodes (N) = 7
- Total Edges (E) = 8
- Connected Components (P) = 1
- Predicate Nodes (Pn) = 2 (Node 1: fee==0, Node 4: discount condition)
- Closed Regions (R) = 2

Verification via 3 Independent Methods:

1. Method 1 (Formula: $E - N + 2P$):

$$V(G) = 8 - 7 + 2(1) = 3$$

2. Method 2 (Formula: $P_n + 1$):

$$V(G) = 2 + 1 = 3$$

3. Method 3 (Formula: Closed Regions + 1):

$$V(G) = 2 + 1 = 3$$

Therefore, exactly 3 linearly independent basis paths exist, requiring a minimum of 3 test cases for 100% path coverage.

6. Derivation of Basis Paths

- Path 1: 1 → 2 (Invalid consultation fee <= 0; returns error code -1.0)
- Path 2: 1 → 3 → 4 → 5 → 6 → 7 (Valid fee, discount applied via insurance or emergency > 5000)
- Path 3: 1 → 3 → 4 → 6 → 7 (Valid fee, no discount applied; returns standard gross bill)

7. Test Case Suite & Execution Results

Test ID	fee	labCost	hasIns	isEmerg	Expected Output	Actual Output	Path Covered	Status
TC-01	0.0	1000.0	false	false	-1.0 (Error)	-1.0	Path 1 (1->2)	PASSED [OK]
TC-02	2000.0	3000.0	true	false	4500.0 (10% off 5000)	4500.0	Path 2 (1->3->4->5->6->7)	PASSED [OK]
TC-03	1500.0	500.0	false	false	2000.0 (No discount)	2000.0	Path 3 (1->3->4->6->7)	PASSED [OK]

8. Conclusion

White-box basis path testing verified all execution paths of the calculateBill() method. Cyclomatic complexity $V(G) = 3$ was satisfied with 100% branch and statement coverage, validating mathematical correctness.